

Microscope Documentation

Contents

1	Microscope Architectural Framework for Salesforce	1
1.1	Introduction	1
1.2	Key Benefits of the Microscope Framework	2
1.3	Microscope Solution	3
1.3.1	Companion Documents	3
1.3.2	Installation	4
1.3.3	Platform Features Utilized	4
1.3.4	Service Model	5
1.3.5	Visibility	8
1.4	Handling Functional Change	11
1.4.1	Service-side Versions	11
1.4.2	Invocation Permission-Based Processing	14
1.5	Operating in Different Contexts	16
1.5.1	Stubs and Alternates	16
1.5.2	Security Benefits	17
1.6	Generative AI Benefits	18

1 Microscope Architectural Framework for Salesforce

1.1 Introduction

Enterprise Salesforce orgs become harder to understand as they grow. Teams lose visibility of how business functions, code, and dependencies connect, and the consequences of change become harder to predict. Delivery slows, release risk increases, and time to market suffers.

This application addresses that problem and solves several others by splitting a complex org's customisations into **business-aligned partitions**. Links between those partitions are **decoupled, abstracted**, and managed through **custom metadata** at an architectural level. Reporting and visualizations over that metadata let teams understand how the org is structured through a macro-level business lens.

1.2 Key Benefits of the Microscope Framework

1. **Org clarity at scale** — Metadata-controlled connections between business-aligned partitions make it possible to understand how functions, code, and dependencies connect at a macro level, reversing the visibility loss that accumulates in large Enterprise orgs.
2. **Decoupled, independently releasable services** — Business logic is abstracted behind a service layer so individual functions can be changed, versioned, and released independently without cascading impact on calling code across Apex, Flow, OmniStudio, or Agentforce.
3. **Concurrent versioning and instant rollback** — New implementations deploy alongside existing ones and are activated via metadata, enabling feature-flagging, phased adoption, zero-downtime upgrades, and one-step rollback without code changes.
4. **Permission-driven variations** — Custom Permissions route different users to different implementations, supporting regional variations, segregated business lines, pilots, and A/B testing from a single configuration model — including for Prompt Templates and Agentforce Actions.
5. **Environment variations** — Switchable stub patterns handle partial-code orgs, disconnected test environments, and production outages through Administrator driven or scripted switching via Custom Settings.
6. **Universal audit trail with call stack** — Every invocation is logged in a consistent, flat structure across the entire org, capturing user, context, permissions, input/output, and errors — with linked parent/child records for composed and asynchronous call chains.
7. **Business intelligence and AI grounding** — The single audit data source can be uploaded to Data 360 and CRM Analytics through configuration-only tooling, enriching the customer view and providing grounding data for Generative AI summarization and predictive functions.
8. **Transaction reruns** — Failed or historic invocations can be replayed directly from the Audit table using the original input and context — a capability unique to this framework on the Salesforce platform.
9. **Generative AI governance end-to-end** — Prompt Templates and Agentforce Actions gain the same service controls as any other invocation: auditing with Einstein Trust Layer safety scores, threshold-based blocking and failover, permission-based prompt routing by region or role, and safe in-production prompt upgrades.
10. **DX packaging and technical debt remediation** — Soft metadata references between caller and service sides allow partial-code deployments in scratch orgs and unlocked packages, complementing Salesforce DX adoption by minimising dependencies and providing a structured path for iden-

tifying and removing obsolete implementations.

Microscope also provides a companion process for architectural delivery that teams need to deliver consistently and predictably.

1.3 Microscope Solution

Microscope is a configuration-driven framework that gives Enterprise Salesforce orgs governance, audit, and runtime control over invocations across Apex, Flow, Agentforce Actions, and OmniStudio. It is designed to make delivery faster and lower-risk while keeping packaging, testing, and rollout hygiene intact.

Five outcomes it supports for delivery programs:

- **Ship smaller, safer, more often** — concurrent service versions, permission-gated pilots, and instant rollback by metadata change rather than redeploy. See [Service-side Versions](#) and [Invocation Permission-Based Processing](#).
- **Let teams move at different speeds** — regional, line-of-business, and package teams can deliver their own service versions without forcing a coordinated org-wide release.
- **Open up enterprise development and testing** — smoother DX and unlocked-package adoption, alternate processing when integrations or LLMs are missing, deterministic substitutes for Gen AI responses where determinism is needed, and graceful degradation in production when connections go down. See [MicroscopeStubs.md](#).
- **Govern Generative AI deployment** — Trust Layer-driven failover and blocking, permission-gated prompt rollout, and regional / jurisdictional Prompt Template variants. See [Generative AI Benefits](#).
- **Audit and analyse, in one shape** — every invocation across Apex, Flow, Agentforce, and OmniStudio is captured in a single structure that can flow into Data 360 and CRM Analytics for ops, security, and usage intelligence. See [Audit and Errors](#).

AI-authoring skills are provided so delivery teams can adopt the patterns without writing the framework's metadata by hand. The full benefit catalogue is in [MicroscopeBenefits.md](#). The rest of this document is the architectural introduction.

1.3.1 Companion Documents

This document describes the core framework. The following companion documents build on it for specific audiences and concerns:

- [MicroscopeAIAgents.md](#) — How the framework supports Generative AI: Prompt Service, prompt safety, role-based prompts, and Gen AI testing patterns. *For architects and developers building Agentforce or LLM-driven features.*

- [MicroscopeEnterpriseDelivery.md](#) — How Microscope addresses enterprise-scale delivery challenges around visibility, governance, and release management. *For executives, programme leads, and enterprise architects.*
- [MicroscopeDeveloperGuide.md](#) — Hands-on patterns and conventions for building on the framework. *For developers and tech leads.*
- [MicroscopeSecurity.md](#) — Permission-based and context-based controls on when and by whom an Invocation can be run. *For architects, security reviewers, and developers exposing functionality to external systems.*
- [MicroscopeStubs.md](#) — Detailed reference for the stub patterns introduced below (Scratch, Absent Service, Absent Connection, Down Status). *For developers and environment managers.*

1.3.2 Installation

Instructions for installing the package are [here](#).

Microscope Skills, to aid AI Code Generation tools, can be downloaded from [here](#). Follow the guidelines for your Coding Agent of choice for how to install these in your development environment (e.g. in a top-level `.claude` folder).

1.3.3 Platform Features Utilized

The solution uses a combination of Lightning Platform features.

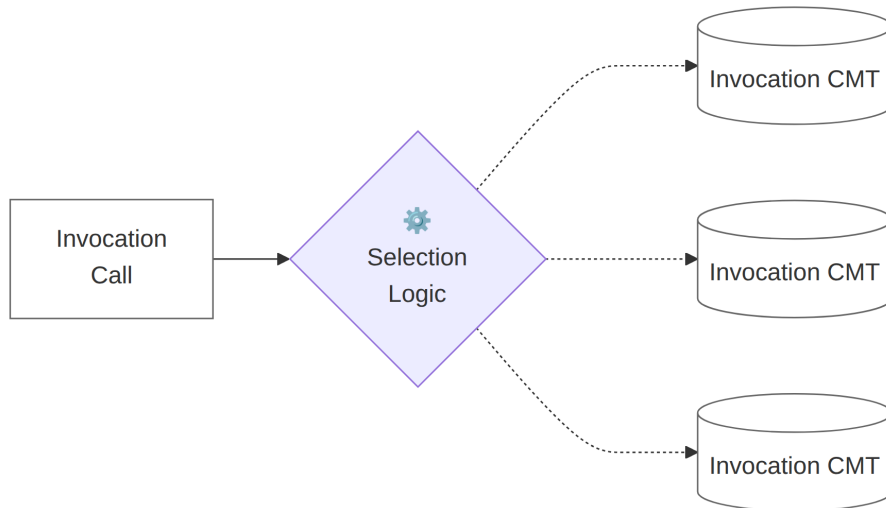
- Custom Metadata Types (*CMTs*) to define the structure of the production build and the mechanisms for special processing in *lower orgs* (scratch orgs and sandboxes).
- Custom Settings for temporary changes in production and to configure features that vary across environments. CMT records are the same across all test environments; Custom Settings are only used when environmental differences occur.
- Custom Permissions to enable differences in processing for different groups of users.
- Custom Objects to capture runtime transaction information for auditing and to allow rerunning of transactions.
- Data 360 for long-term storage of audit records of user interactions.
- CRM Analytics for visualizations and reports of audit data and build structure.
- Platform Events for asynchronous communication between functional components and auditing services.
- Platform Cache to reduce calls to Custom Metadata and maintain high processing performance.

1.3.4 Service Model

The framework is built around a configuration-driven service model using Custom Metadata Types.

At runtime the model works like this, details will follow:

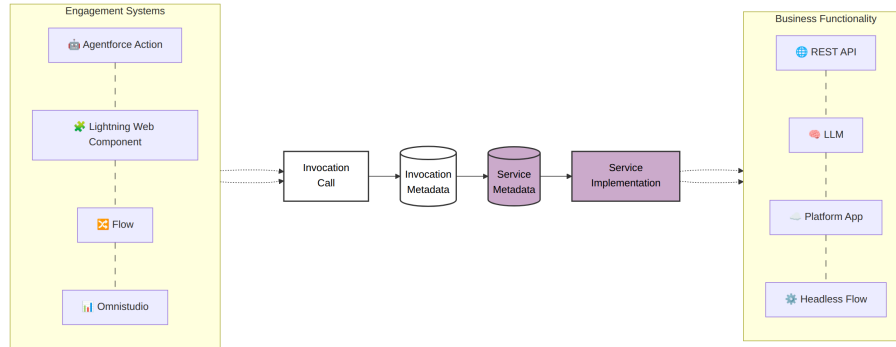
1. A caller references an **Invocation Call** value. This value must be used only once in calling code across the org. The calling technique varies by technology.
 - In Apex the *Invocation Call* field is used as an input argument to a method that initializes an instance of a Service Invocation class.
 - In Flow the *Invocation Call* field is an input parameter to an Invocable Method
 - In OmniStudio tools the *Invocation Call* field is used as a map parameter.
2. The runtime framework uses this *Invocation Call* value to retrieve one or more related *Invocation* CMT records and depending on the running context, one is selected. *Invocation* CMT records can optionally reference custom permissions to provide different processing for different users (depending on the custom permissions they have assigned) which drives the selection of the correct *Invocation* CMT record. The full selection logic is described in [Invocation Permission Selection](#).



3. The selected *Invocation CMT* record contains fields such as *Service Name*, *Service Version*, *Method Name*, *Method Iteration*, and *Implementation Version*. Together, those values identify the target Service-side records.
4. Using the *Service-side* metadata, the framework resolves the correct *Implementation*, such as an Apex class or a Flow.

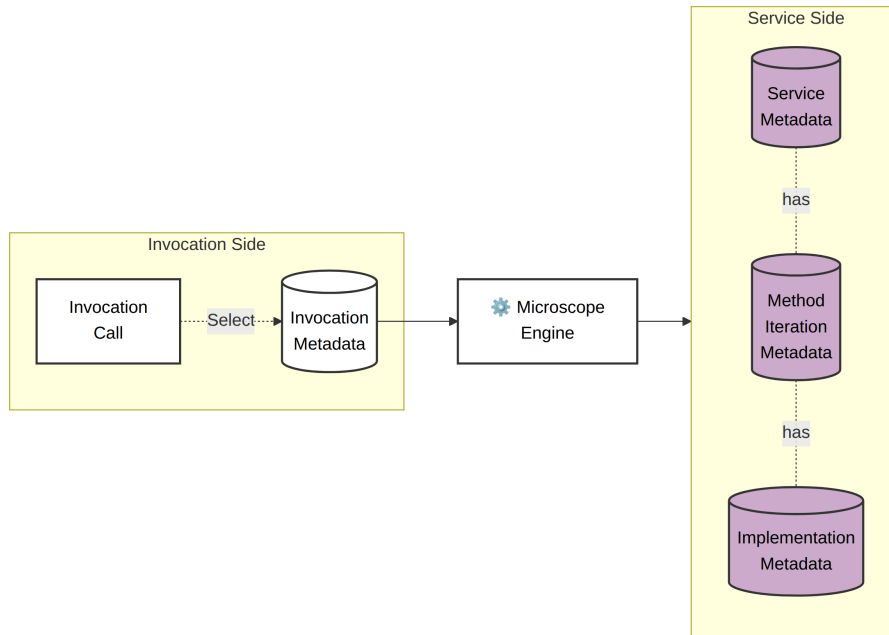
5. The framework runs the self-contained implementation, which performs the business functionality, delegates to other applications or interacts with external systems like LLMs, and returns the result and status back to the caller.

In the diagram below, the left-hand side is the **Invocation-side**. Engagement systems such as page components, Flows, and Agentforce Actions. The right-hand side is the **Service-side** which facilitates the business functionality.



The Engagement systems on the left reference **Invocation CMT** records which abstract the business functionality from the caller. These CMT records reference 3 tiers of metadata on the *Service-side*.

- High-level business capabilities are represented by **Service CMT** records. A Service might represent an external interface, abstract a large package, or encapsulate a custom platform capability developed by the Enterprise.
- The individual functions exposed by a Service are represented by **Method Iteration CMT** records. These records define the method **signature**, the input and output definitions.
- **Implementation CMT** records point to what actually runs, such as an Apex class or a Flow. Each runnable artifact should be self contained with references to only the bare minimum needed to implement the functionality.



There are deliberately no direct lookup fields between the left-hand side and the right-hand side of the model. In development and test scenarios, teams may want to deploy an *Invocation CMT* record into a **partial-code** environment (like a scratch org) where the related Service-side CMT records are not yet present. Instead, the Service, Method, and Implementation names and versions on the *Invocation CMT* act as “soft” lookups. When the relevant Service-side records exist (as they always will in a **full-code** environment, i.e. one with the full Enterprise code-base installed), those values uniquely identify the Implementation CMT record to use.

The section on [Invocation Permissions](#) explains what can be done when the Service-side records are absent.

Invocations can run synchronously or asynchronously, using Platform Events or Queueables depending on the requirement. Throughout the process, the framework tracks *Invocation Details*. Errors are handled consistently as part of that same structure.

To avoid repeated queries, the Service-side field values used for an Invocation can be stored in Platform Cache for reuse. The cache is refreshed whenever an *Invocation CMT* record changes.

Invocations of Services may be composed by *embedding* a new call to an Invocation inside a Service Implementation, or by *chaining*: using the output of one or more Services as the input to another Invocation using Flows or Apex.

The Service Model is the foundation of the framework. It standardizes how

invocations are made and how services are implemented across multiple platform technologies. Because the connections are controlled by metadata, they become visible and governable. Services also provide a layer of abstraction: they hide implementation complexity from callers, expose only what teams choose to expose, and let Enterprises apply their own business vocabulary over vendor capabilities. This reduces dependency coupling and helps limit vendor lock-in.

To implement these patterns, you can either run the relevant AI skills or check out the documentation that comes with the skills.

- **Setup a new Service method:** [AI Skill](#) | [Documentation](#)
- **Setup a new caller-side Apex invocation:** [AI Skill](#) | [Documentation](#)
- **Refactor an existing Apex method into a Service:** [AI Skill](#) | [Documentation](#)

1.3.5 Visibility

The framework provides visibility into how parts of the org are connected, via CMT configuration objects, and how the org is used, via Audit records. This feeds into both structural analysis, which is useful for delivery teams and code generation, and usage analysis, which is useful for business intelligence and AI grounding.

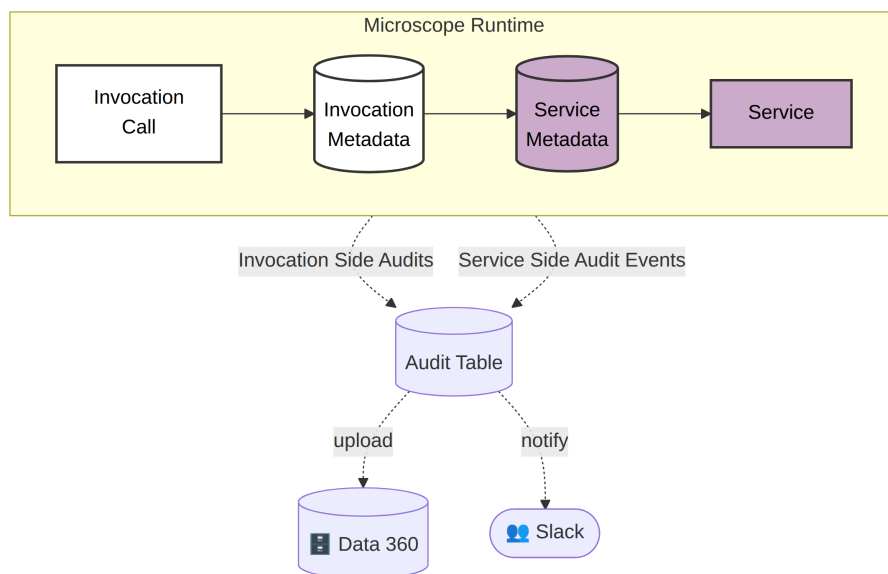
Both Structural and Usage data items can be easily uploaded to Salesforce Data 360 and CRM Analytics using configuration-only tooling.

1.3.5.1 Structural Analysis Insights Making connections visible in an org through Invocation and Service-side metadata provides clarity on how changes should be implemented and gives delivery teams a common vocabulary.

- Configuration metadata gives a view of all the invocations and related implementations in the org, increasing understanding of the org's current state for teams involved in delivery.
- High Level functional visualizations allow analysts and architects to add details about implementations to their designs to ensure that the build matches the design.

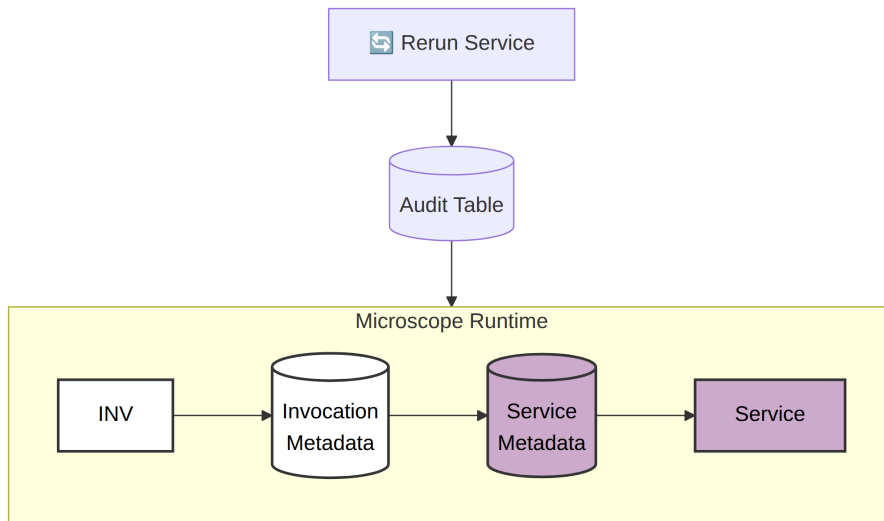
1.3.5.2 Audit and Errors Every runtime invocation creates an **Invocation Details** object instance. It captures information about the invocation, including data derived from the Invocation and Service-side CMT records, plus runtime values such as the current user, the Custom Permission that allowed the invocation, execution context, and timestamps. The framework passes *Invocation Details* into the implementation, which can enrich it with additional data such as processing context or the final Success or Failure status. A standard error structure is part of *Invocation Details* and is populated consistently in both Flow and Apex.

The *Invocation CMT* can instruct the framework to save *Invocation Details*, together with serialized input and output data, to a custom object named **Audit**. This can happen at two points: when the invocation starts, and after the implementation finishes. Audit can run either synchronously, with records saved in the same transaction, or asynchronously.



The framework’s transaction audit has a simple, flat structure that is well suited to error detection, alerting, and upload into analytics platforms. It also supports multi-level auditing of related invocations. If one parent invocation runs an embedded child invocation inside its Service implementation, the child’s audit record links back to the parent, providing an **Invocation Call Stack**.

Invocations may be **rerun** from the Audit Table by using the *Invocation Details* and the serialized input data to recreate the original invocation.



The Audit table also provides runtime benefits.

- Full Transaction Audit and error flagging performed in a common way across all business functions and all orgs allows error handling and alerting to be performed via a single mechanism and for tools to be written to enhance the error handling across all customers using the framework.
- Full **Linked Audit of Asynchronous events**. Each invocation is stamped with a unique id. If the invocation is asynchronous, that id is passed through to the Service implementation so that Service-side enrichment and output is linked back to the original invocation record in the Audit table.
- An alerting framework, for example one integrated with Slack, can be layered on top of Audit and triggered for defined subsets of records such as errors with specific error codes.

Note that Auditing is optional in the framework and is controlled at the invocation level by the *Invocation CMT* records. High data volumes or sensitive data are examples that might lead teams to choose not to audit.

This single audit shape — one consistent structure across Flow, Apex, Agentforce, and OmniStudio invocations — is the substrate that powers operations, security, and AI grounding from the same source.

The details of the Audit process can be found [here](#)

1.3.5.3 Usage Analysis and Data Grounding The single audit framework across the org provides a huge amount of information in a common format that can be ingested by data and visualizations platforms. For example uploads to Data 360 and CRM Analytics can be achieved using standard tooling and

can power engagement analysis and click history for internal and external users across LWC and Flows. This can be used as input to Predictive AI functions and as grounding for Gen AI prompting.

Runtime Intelligence on Usage, Performance, Forensic and Trend intelligence can be made available in Data 360 or CRM Analytics for Business, Delivery, Operations and Security Teams at transactional and aggregates levels.

1.4 Handling Functional Change

1.4.1 Service-side Versions

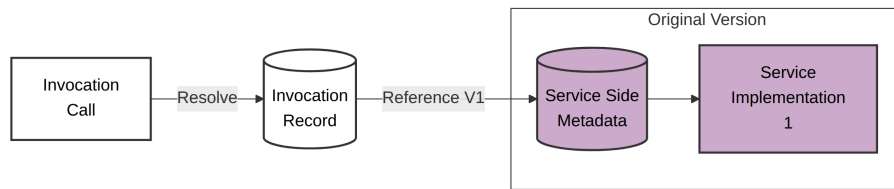
Each of the Service-side CMTs (Service, Method, Implementation) in the Data Model has a version number field and different versions can exist concurrently at any of these levels. The semantics for the 3 version numbers are:

- **Service Version:** Any Service can be versioned. The *Service CMT* has only a small number of fields and is not versioned in the same way as Method and Implementation and is only mandated for large releases of functionality. Service versioning is best tied to namespaces. A Service Version and all its related artefacts (methods, implementations etc) can be deployed in a namespaced package. Artefacts like Apex class names can then be reused multiple times in different Service Versions within the same org. The Service Version should increment whenever a new namespace is used and the namespace ensures unique Fully Qualified Names.
- **Method Iteration:** The *Method Iteration CMT* specifies the method signature (the input and output). If the signature changes there should be a new iteration of the Method.
- **Implementation Version:** This increments when there is a change in an Implementation artefact for the same Method Iteration, the signature has not changed but the implementation has.

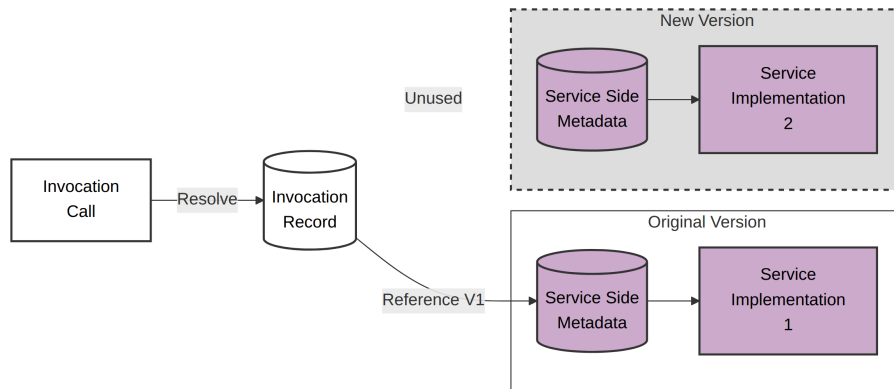
The framework promotes the use of new, concurrent *Implementation Versions* in preference to editing what is already deployed (a *No Edit principle*). An *Implementation Version* should be considered as immutable, they can be patched if an urgent fix is required but otherwise should not change, but be replaced with a new version when changes are required.

This is how change is typically implemented to provide small, fast, safe releases.

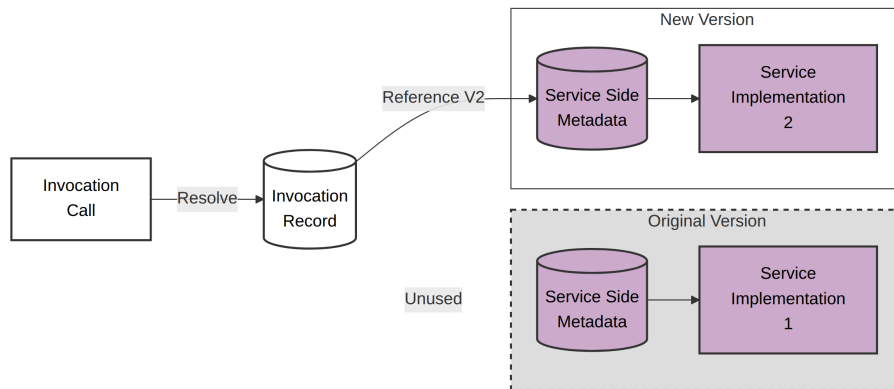
1. Starting Position - invocation points to Implementation *V1*, a predominantly self-contained flow or Apex class.



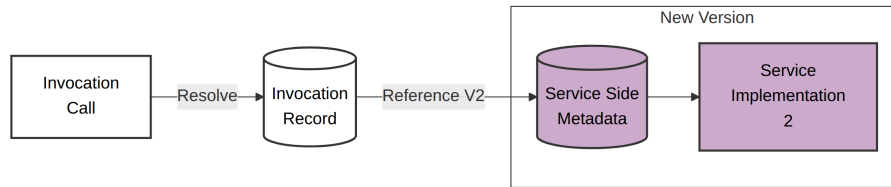
2. Version Added - Implementation $V2$ is added, but the invocation does not need to adopt it immediately



3. Version Adopted - Implementation $V2$ is adopted but $V1$ can remain, either for rollback assurance or if other invocations are still using it



4. End Position - Implementation $V1$ is removed when safe to do so



The *Invocation CMT* has references to the names and versions of Service-side CMTs. The team that owns an *Invocation CMT* record can switch to using a new combination of *Service Version*, *Method Iteration* or *Implementation Version* by updating these references. If a particular *Method Iteration* is invoked from multiple places, concurrent versions help to avoid the need for all of these to change their functionality at the same time.

The ability to run concurrent Service-side Versions at various levels provides these benefits:

- **Concurrent Versions** : No Edit principle, with new self-contained versions rather than modifications to existing ones,, supporting concurrent live versions.
- **Feature Flagging** : Immediate deployment of Service-side versions with later activation (Service-side feature flagging).
- **Phased Version Adoption** : Service-side Versions prevent enforced simultaneous functionality change across the whole org when a service used by multiple parts of the business is upgraded.
- **Rollback** : Releases can be rolled back by reverting Invocation metadata records to point to previous versions. The older Service-side code and metadata can remain in the org until the new version has safely bedded in.
- **Minimized Build Dependencies**, Each *Invocation Call* should be referenced in **only one place in the build**. Referencing the same *Invocation Call* from multiple places would create an unnecessary link between those parts of the build - avoiding shared Invocation metadata record reduces dependencies and maximizes the ability to change functionality independently using metadata changes only, not code.
- **Build Hygiene** : Over time, an *Implementation Versions* may be superseded become obsolete, with no invocations calling it. Those that are no longer called can be recognised by running a simple report on the Invocation metadata records. The Service-side code artefacts and CMT records identified as redundant can then be removed.
- **Always use Latest Version** : Although we said earlier an *Invocation* must provide an *Implementation Version* value, this can be left blank. If so the runtime framework will choose the *Implementation CMT Record* with the highest *Implementation Version* for the *Method Iteration*.

To implement this pattern, you can either run the relevant AI skills or check out the documentation that comes with the skills.

- **Create a new Method Iteration:** [AI Skill](#) | [Documentation](#)
- **Create a new Implementation Version:** [AI Skill](#) | [Documentation](#)

1.4.2 Invocation Permission-Based Processing

(Different behavior for invocations for different users)

As mentioned, calling applications invoke the framework by referencing an *Invocation Call*. This value should match the *Invocation Call* field of at least one *Invocation CMT* record. It may match more than one record because the *Invocation CMT* also has an optional field named *Invocation Permission*, which stores the API name of a Custom Permission. Records with this field populated are called **Permission Overrides**.

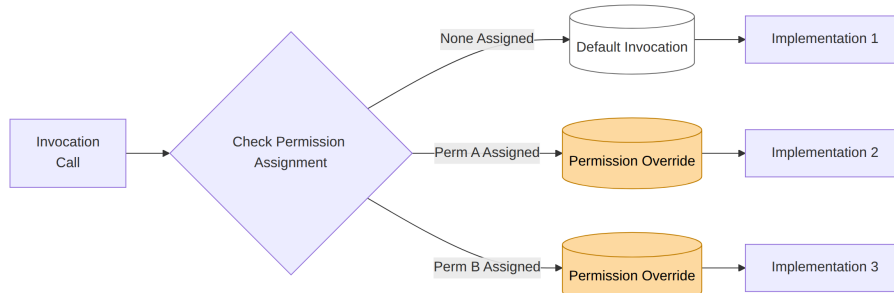
When an invocation is executed, the framework initially checks all Permission Override records with the referenced *Invocation Call* value.

- If the calling user has one of the referenced Custom Permissions assigned, the matching *Invocation CMT* record is used.
- If none of the populated *Invocation Permission* values match the user's permissions, the record with a blank *Invocation Permission* field is used. This is the *Default Invocation CMT Record*.

Together, the *Invocation Call* and *Invocation Permission* fields form a unique key for the *Invocation CMT*. The calling code references only the *Invocation Call* but not the *Invocation Permission*. This allows teams to change behavior for different user groups without changing the code.

The setup is simple:

- Create a new Custom Permission
- Assign that to a subset of users. To ensure consistency and clarity of behaviour make sure that no user is assigned more than one Custom Permission for any *Invocation Call*.
- Add a new *Invocation CMT* record with this Custom Permission referenced in the *Invocation Permission* field.



Invocation Permissions can be used for both temporary and permanent situations, from pilot implementations and hotfixes to regional variations or any other case where segmented groups of users need different processing.

1.4.2.1 Functional Pilots Pilot runs of new Service-side Versions for subsets of users can be provided via *Invocation Permissions*. Teams develop the pilot implementation and when it is ready to deploy create a new custom permission and a new Permission Override CMT record that references the permission. The Override then routes the pilot users to the pilot implementation. If Invocation Permissions are already used to provide different functionality for different users, a second Invocation Permission layer may be required, and the field *Invocation Permission 2* can be used for this (see [Invocation Permission Selection](#)).

If the Pilot is successful, its *Invocation CMT* record can become the new default by clearing the *Invocation Permission* field and deleting the previous default record.

How to Implement: If you wish to implement this pattern, you can either run the relevant AI skill or check out the documentation that comes with the skill.

- **Create a Functional Pilot:** [AI Skill](#) | [Documentation](#)

1.4.2.2 Segregated User Functionality Invocation Permissions can be used to provide different long-term functionality for different user groups. For example, if users from different countries should use different pricing services, we can assign a custom permission to the designated users in each country, create Permission Override CMT records referencing each country permission, and route those users to the appropriate implementation.

If two levels of refinement are required, for example per country / per line of business, the *Invocation CMT* field **Invocation Permission 2** can be used. For the complete multi-permission selection process, see [Invocation Permission Selection](#).

1.4.2.3 Hotfixes Emergency Hot Fixes can be tested by a small group of users assigned a specified custom permission prior to rolling out to the full user

base. These can be considered as very quick, unscheduled pilots, and the steps are the same.

- Deploy a new Service-side Implementation Version record pointing to the fix implementation.
- Create a new *Permission Override* record with the same *Invocation Call* value which references a new Custom Permission and points to the new implementation.
- Create and assign that permission to the team testing the fix

The hotfix can become the default live version by clearing the *Invocation Permission* field and removing the original *Default Invocation CMT* record.

Technically, segregated user functionality and hotfixes are identical to functional pilots, so the same implementation links apply.

1.5 Operating in Different Contexts

Ideally, an agile enterprise will not have developers and testers always working in *full code* monolithic environments, but in smaller *partial* environments. The advantages are well understood but a number of challenges have to be addressed, principally concerning how to develop in environments where artefacts that will be called in production are not actually available or fully configured.

Further along the path to production, environments may have a full code base but data, or connections to external systems, are not complete or available. The behaviour in these environments is often different from production and this makes testing difficult.

Further challenges arise in production environments where different processing is required at different times, for example during a maintenance window or when a connection is unavailable.

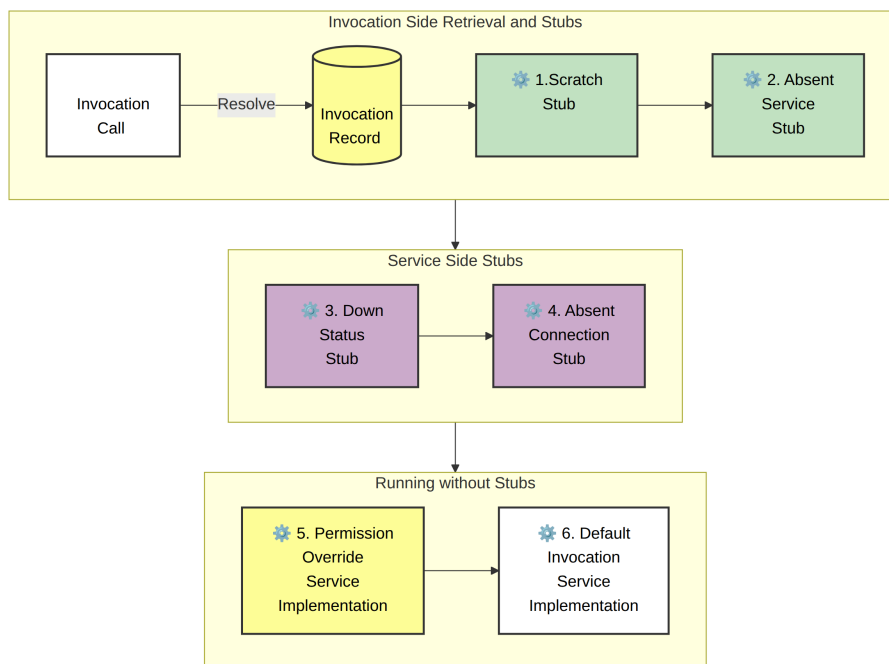
1.5.1 Stubs and Alternates

Microscope handles partial environments, missing connections, and production safety scenarios through four **stub patterns**. A stub provides an alternate implementation that runs in place of the real Service Implementation, configured through metadata and Custom Settings rather than code branches.

Pattern	Side	Environment	Triggered By	Primary Use Case
Scratch Stub	Invocation	Partial code	Custom Setting present	Early development before a real implementation exists
Absent Service Stub	Invocation	Partial code	Service CMT absent in org	DX / unlocked package separation

Pattern	Side	Environment	Triggered By	Primary Use Case
Absent Connection Stub	Service	Full code	Custom Setting present	Test envs missing integrations / Gen AI determinism / token cost
Down Status Stub	Service	Full code	Custom Setting (manual or programmatic)	Production maintenance / outage / Gen AI safety blocking

When an Invocation runs, the framework checks for stubs in a defined order. Invocation-side stubs take precedence over Service-side stubs, because they short-circuit the dispatch before service resolution.



For per-pattern detail — including decision guidance on which pattern to pick, anti-patterns, and stubs in unit testing — see [MicroscopeStubs.md](#).

1.5.2 Security Benefits

Microscope provides permission-based and context-based mechanisms to restrict when and by whom an Invocation can be run. See [Microscope Security](#) for the full details, including quiddity reference tables, unit test exemptions, and guidance on exposing invocations to external systems.

1.6 Generative AI Benefits

The framework’s metadata-driven dispatch, permission-based variants, and Service-side stub patterns turn out to be exactly the primitives needed for enterprise-grade Gen AI deployment:

- **Prompt safety** — Trust Layer scores can drive failover, threshold-based blocking, and “recent safest” template selection across a configured group of Prompt Templates.
- **Permission-gated prompt rollout** — pilot a new Prompt Template to a subset of users via Custom Permissions, without forking or overwriting the existing template.
- **Regional / jurisdictional variants** — different LLMs, or no LLM, for different countries, controlled by metadata rather than code.
- **Test-environment grounding** — production-like retrieval payloads via stubs and deterministic LLM responses for business-process testing, with a side benefit of reduced token consumption.
- **Trust Layer-enhanced audit** — every Prompt Template run is captured with safety, toxicity, and bias scores against user context, input, and output.

For the full Gen AI architecture — including the Prompt Service, failover algorithms, and prompt change management workflow — see [MicroscopeAIAgents.md](#).